

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA AN TOÀN THÔNG TIN

BÁO CÁO CHUYÊN ĐỀ

An toàn Thông tin

Đề tài

**Phát hiện tấn công leo thang đặc quyền
trong Kubernetes dựa trên lời gọi hệ thống**

*Nghiên cứu mô-đun phát hiện bất thường SyscallAD
của framework DeSFAM*

Giảng viên hướng dẫn : TS. Vũ Thị Vân

Học viên thực hiện : Nguyễn Ngọc Hoài — CHAT4P006

Lớp : CHAT4P

Hà Nội, tháng 6 năm 2026

Tóm tắt

Trong môi trường Kubernetes, các container chia sẻ chung một nhân hệ điều hành; do đó *lời gọi hệ thống* (syscall) trở thành bề mặt tấn công trọng yếu cho *leo thang đặc quyền* và thoát container. Báo cáo chuyên đề này **ngiên cứu mô-đun phát hiện bất thường SyscallAD** (gọi tắt SysAD) của framework DeSFAM [1]—một framework giám sát và bảo vệ container dựa trên syscall gồm ba mô-đun: (1) lập danh sách truy cập syscall lai và sinh hồ sơ seccomp; (2) *SyscallAD*—giám sát thời gian thực và phát hiện bất thường bằng tập hợp Bộ mã hóa biến phân (VAE) và Rừng cô lập (iForest); (3) thực thi thích ứng trong nhân. Trọng tâm của báo cáo là **mô-đun thứ hai (SyscallAD)**: cơ chế biến chuỗi syscall thành đặc trưng, mô hình học máy phát hiện bất thường, cách hợp nhất điểm và chọn ngưỡng, cùng cách mô-đun này nắm bắt *dấu vết hành vi* của leo thang đặc quyền (sự hiện diện của các syscall đặc quyền như `ptrace`, `setuid/capset`, `setns`, `unshare`, `mount`, `bpf`, nạp mô-đun nhân). Hai mô-đun còn lại chỉ được *điểm qua* để giữ tính trọn vẹn của bức tranh kiến trúc.

Để kiểm chứng, chúng tôi tái lập SyscallAD và huấn luyện trên bộ dữ liệu công khai DongTing [2], rồi đánh giá *hoàn toàn ngoại tuyến* (huấn luyện → kiểm tra → đánh giá). Toàn bộ quy trình được đóng gói bằng Docker để bảo đảm khả năng tái lập. Trên bộ DongTing đầy đủ (18 874 chuỗi nạp được trên 18 966 công bố), lỗi phát hiện đạt AUC tập hợp 0,835 (AP 0,990, precision 0,999) ở mức cửa sổ và 0,787 ở mức chuỗi. (DeSFAM không công bố bảng số theo mô hình trên DongTing, nên báo cáo chỉ tham chiếu định tính, không tuyên bố “đạt/vượt số liệu công bố”.) Một thí nghiệm cắt bỏ cho kết luận phân đôi đáng chú ý: ở mức cửa sổ, một bộ đếm hiện diện syscall đặc quyền (AUC 0,875) tương đương mô hình đầy đủ—tức phần lớn năng lực phát hiện quy về “có syscall đặc quyền hay không”; nhưng ở mức chuỗi, mô hình học máy mới bộc lộ giá trị tăng thêm (0,787 so với 0,678). Báo cáo đối chiếu trung thực với DeSFAM (lưu ý rằng DeSFAM không công bố bảng số theo mô hình) và rút ra bài học về vai trò của chọn lọc đặc trưng theo miền cùng đơn vị đánh giá (cửa sổ so với chuỗi).

Từ khóa: leo thang đặc quyền, bảo mật container, Kubernetes, lời gọi hệ thống, phát hiện bất thường, SyscallAD, DeSFAM, Bộ mã hóa biến phân (VAE), Rừng cô lập (iForest), eBPF, Tetragon, DongTing.

Mục lục

1	Mở đầu	1
1.1	Bối cảnh và động cơ	1
1.2	Đối tượng và phạm vi nghiên cứu	1
1.3	Mục tiêu nghiên cứu	1
1.4	Đóng góp	2
1.5	Bố cục	2
2	Cơ sở lý thuyết	3
2.1	Leo thang đặc quyền và thoát container trong Kubernetes	3
2.2	Giám sát lời gọi hệ thống cho container	3
2.3	Phát hiện xâm nhập dựa trên chuỗi syscall	3
2.4	Công trình gần đây (2023–2026)	4
2.5	Bộ mã hóa biến phân (VAE) và Rừng cô lập (iForest)	4
2.6	Bộ dữ liệu DongTing	4
3	Tổng quan framework DeSFAM	4
3.1	Kiến trúc ba mô-đun	4
3.2	Mô-đun 1 — Lập danh sách truy cập syscall lai và hồ sơ seccomp (điểm qua)	5
3.3	Mô-đun 3 — Thực thi thích ứng và quản lý chính sách (điểm qua)	5
3.4	Vị trí của SyscallAD và lý do chọn làm trọng tâm	6
4	Mô-đun SyscallAD — kiến trúc và thuật toán	6
4.1	Tổng quan đường ống phát hiện	6
4.2	Thu thập, tiền xử lý và “bảng chữ cái syscall an ninh”	7
4.3	Trích đặc trưng theo cửa sổ trượt	7
4.4	Cơ sở dữ liệu mẫu lành tính bằng PrefixSpan	7
4.5	Mô hình thành phần	8
4.6	Hợp nhất tập hợp	8
4.7	Chọn ngưỡng	8
4.8	Thuật toán phát hiện	9
5	Cơ chế phát hiện leo thang đặc quyền của SyscallAD	9
5.1	Ánh xạ syscall đặc quyền sang kỹ thuật leo thang	9
5.2	Vì sao đặc trưng disc là tín hiệu cốt lõi	11
5.3	Ví dụ minh họa một chuỗi tấn công	11

6	Thiết lập thực nghiệm	11
6.1	Nguyên tắc: mọi công cụ chạy bằng Docker	11
6.2	Huấn luyện ngoại tuyến SyscallAD	12
6.3	Cấu hình thí nghiệm (khai báo trước)	12
6.4	Giao thức đánh giá và tiêu chí tái lập thành công	12
6.5	Dữ liệu tấn công và lành tính	14
7	Kết quả và quan sát	14
7.1	Hiệu lực phát hiện ngoại tuyến trên DongTing	14
7.2	Tham chiếu định tính tới DeSFAM	15
7.3	Cấu hình bảng chữ cái syscall an ninh	15
8	Thảo luận	16
8.1	SyscallAD đạt được gì so với công trình tham chiếu	16
8.2	Ý nghĩa của khoảng cách ngưỡng và quản trị ngưỡng khi vận hành . . .	16
8.3	Lựa chọn thiết kế và phạm vi	16
8.4	Phản biện về ảnh hưởng của việc thu hẹp bảng chữ cái syscall an ninh .	16
8.5	Hạn chế và đe dọa tính hợp lệ	17
9	Kết luận và hướng phát triển	19
A	Phụ lục: Tái lập kết quả	22
A.1	Chuẩn bị dữ liệu DongTing đầy đủ	22
A.2	Huấn luyện ngoại tuyến (sinh số cho Bảng 5)	22
A.3	Kiểm tra tính nhất quán đặc trưng (train $\leftrightarrow$ serve)	22
A.4	Biên dịch báo cáo này (Docker)	22

Danh mục hình vẽ

1	Kiến trúc ba mô-đun của DeSFAM. Báo cáo này đặt trọng tâm vào Mô-đun 2 (SyscallAD)—phát hiện bất thường syscall; Mô-đun 1 và Mô-đun 3 chỉ được điểm qua.	5
2	Đường ống phát hiện của SyscallAD: lọc theo bảng chữ cái syscall an ninh $\rightarrow$ cửa sổ trượt (15,3) $\rightarrow$ véc-tơ 43 chiều, chấm điểm song song bằng VAE (lỗi tái cấu trúc) và Isolation Forest (độ sâu cô lập), hợp nhất robust ($\alpha = 0,7$), so với ngưỡng T_0 để phát cờ. Cả VAE và iForest đều huấn luyện <i>chỉ trên cửa sổ lạnh tính</i>	6
3	Kỹ thuật đặc trưng của SyscallAD: từ chuỗi syscall thô $\rightarrow$ lọc theo bảng chữ cái syscall an ninh $\rightarrow$ cửa sổ trượt (15,3) $\rightarrow$ véc-tơ 43 chiều.	8
4	Ánh xạ các syscall đặc quyền (đặc trưng disc) sang nhóm hành vi leo thang đặc quyền và ví dụ CVE thực. Bit hiện diện của một syscall đặc quyền là tín hiệu phát hiện cốt lõi của SyscallAD.	10
5	Hiệu năng mức cửa sổ của ba mô hình trên tập kiểm tra DongTing. AP cao đồng đều ($\approx 0,99$) và AUC cao (0,73–0,85) phản ánh khả năng <i>xếp hạng</i> tốt; F1 thấp (0,42–0,47) là hệ quả của ngưỡng p995 ưu tiên precision (recall 0,26–0,31).	15
6	Thí nghiệm cắt bỏ trên tập kiểm tra DongTing: AUC của tập hợp đầy đủ (VAE + iForest) so với bộ phân loại chỉ dùng đặc trưng disc . Ở <i>mức cửa sổ</i> , chỉ- disc (0,875) ngang bằng hoặc cao hơn mô hình đầy đủ (0,835)—phản biện “đường tắt theo hiện diện” có cơ sở. Ở <i>mức chuỗi</i> , mô hình đầy đủ (0,787) vượt rõ chỉ- disc (0,678)—mô hình học máy đóng góp giá trị vượt trên hiện diện đơn thuần.	18

Danh mục bảng

1	Một số họ lỗ hổng leo thang đặc quyền/thoát container và dấu vết syscall.	3
2	Ánh xạ syscall đặc quyền $\rightarrow$ hành vi leo thang $\rightarrow$ kỹ thuật MITRE ATT&CK / ví dụ CVE.	10
3	Cấu hình thí nghiệm đã khai báo cho SyscallAD.	12
4	Trung thành với DeSFAM so với thích nghi trong bản tái lập.	13
5	Hiệu năng SyscallAD trên tập kiểm tra DongTing đầy đủ (cửa sổ 15/3, 43 chiều). Báo cáo ở <i>cả</i> hai mức cửa sổ và chuỗi (Mục 6.4); hai hàng cuối là ablation “chỉ disc ” (Mục 8.4).	14

1 Mở đầu

1.1 Bối cảnh và động cơ

Kubernetes đã trở thành nền tảng điều phối container phổ biến nhất cho điện toán đám mây hiện đại. Khác với máy ảo, các container trên cùng một nút (node) *chia sẻ chung một nhân* hệ điều hành; ranh giới cô lập giữa chúng vì thế mỏng hơn nhiều và hội tụ tại một điểm duy nhất: giao diện lời gọi hệ thống (syscall)—ranh giới giữa không gian người dùng và nhân [3, 4]. Một nhân Linux phơi bày hàng trăm syscall, phần lớn không cần thiết cho một khối lượng công việc cụ thể, khiến bề mặt tấn công phình to [5].

Hệ quả là *leo thang đặc quyền* (privilege escalation) và *thoát container* (container escape) trở thành lớp mối đe dọa nguy hiểm nhất: một container bị xâm phạm có thể lạm dụng syscall đặc quyền để chiếm quyền root trên nút và phá vỡ cô lập [6]. Các họ lỗ hổng tiêu biểu—PwnKit (CVE-2021-4034) [7], Dirty Pipe (CVE-2022-0847) [8], thoát runc (CVE-2019-5736) [9]—đều để lại *dấu vết hành vi* ở tầng syscall (gọi ptrace, setuid/capset, unshare/setns, mount, nạp mô-đun nhân, bpf...). Các cơ chế tĩnh như hồ sơ seccomp khó duy trì, dễ chặn nhầm thao tác hợp lệ hoặc bỏ sót tấn công nhiều bước, và không nhận thức được *chuỗi* hành vi đặc trưng cho leo thang đặc quyền.

Vì vậy, một hướng tiếp cận thích nghi—*phát hiện bất thường* trên chuỗi syscall thời gian thực—ngày càng được quan tâm. Framework DeSFAM [1] là một đại diện tiêu biểu: nó kết hợp lập hồ sơ syscall lai, phát hiện bất thường bằng học máy độ trễ thấp, và thực thi trong nhân qua eBPF/LSM. Trái tim phát hiện của DeSFAM là mô-đun *SyscallAD*—đối tượng nghiên cứu của báo cáo này.

1.2 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của báo cáo là *mô-đun phát hiện bất thường SyscallAD* (SysAD) trong framework DeSFAM. Framework DeSFAM gồm *ba* mô-đun (giai đoạn) tích hợp:

Mô-đun 1. Lập danh sách truy cập syscall lai và sinh hồ sơ seccomp (Giai đoạn 1): tĩnh + động + lọc theo CVE.

Mô-đun 2. SyscallAD—giám sát thời gian thực và phát hiện bất thường (Giai đoạn 2): tập hợp VAE + iForest trên chuỗi syscall theo cửa sổ trượt.

Mô-đun 3. Thực thi thích ứng và quản lý chính sách (Giai đoạn 3): chấm điểm rủi ro, ánh xạ MITRE ATT&CK, chặn trong nhân.

Phạm vi: báo cáo *tập trung* vào Mô-đun 2 (SyscallAD)—cơ chế phát hiện leo thang đặc quyền dựa trên hành vi syscall. Mô-đun 1 và Mô-đun 3 *chỉ được điểm qua* (Mục 3) nhằm định vị SyscallAD trong tổng thể kiến trúc, *không* phải đối tượng nghiên cứu chính và không được hiện thực hóa lại. Ngữ cảnh ứng dụng được chọn là phát hiện leo thang đặc quyền trong Kubernetes.

1.3 Mục tiêu nghiên cứu

Là một báo cáo học thuật xây dựng trên một công trình đã công bố (DeSFAM), mục tiêu của báo cáo *không* phải đề xuất một phương pháp mới mà là *nghiên cứu và tái lập*

mô-đun SyscallAD nhằm:

- (i) hiểu và hệ thống hóa thiết kế của SyscallAD—cơ chế biến chuỗi syscall thành đặc trưng, mô hình học máy, hợp nhất điểm và chọn ngưỡng—cùng cách nó nắm bắt dấu vết hành vi của leo thang đặc quyền;
- (ii) *tái lập trung thực* SyscallAD và chạy lại thực nghiệm trên DongTing để **cung cấp bằng chứng sơ bộ về tính khả thi** (feasibility)—trong phạm vi bộ dữ liệu và kịch bản *mô phỏng* đã dùng—rằng cơ chế phát hiện vận hành *phù hợp* với công bố, với quy trình đóng gói bằng Docker có thể chạy lại độc lập. Báo cáo phân biệt rõ “*tái lập trung thành với nguồn*” (cùng pipeline, cùng bậc số liệu) với “*chứng minh cơ chế đúng*”: cái sau cần thí nghiệm cắt bỏ ở Mục 8.4 và do đó chỉ được kết luận thận trọng;
- (iii) tạo một nền tảng tái lập được, làm cơ sở cho các nghiên cứu mở rộng về sau (bổ sung đặc trưng, mở rộng khối lượng công việc, kết nối khâu thực thi).

1.4 Đóng góp

So với việc chỉ *lập lại* DeSFAM, báo cáo có ba đóng góp cụ thể:

- (1) một bản *tái hiện độc lập* mô-đun SyscallAD kèm bộ tạo phẩm đóng gói bằng Docker, chạy lại được;
- (2) một *bộ công cụ đánh giá nghiêm ngặt* (đóng gói trong Experiment/): chỉ số mức cửa sổ và mức chuỗi, thí nghiệm cắt bỏ để kiểm tra nhiều thiết kế, kiểm toán mã hóa, và tiêu chí đánh giá đặt trước—những thứ đường ống gốc thiếu.

Phạm vi báo cáo này: hoàn tất *đường ống ngoại tuyến* (huấn luyện → kiểm tra → đánh giá) của mô-đun SyscallAD trên DongTing. Việc phục vụ/thực thi trực tiếp (triển khai trên hạ tầng container) *không* thuộc phạm vi báo cáo.

Giới hạn mô hình đe dọa. Báo cáo chỉ xét leo thang đặc quyền/thoát container *quan sát được ở tầng syscall, mức một nút*. Các con đường leo thang ở *tầng điều phối* (đánh cắp token ServiceAccount, RBAC quá rộng, lạm dụng dịch vụ metadata) phần lớn *vô hình* ở tầng syscall mức nút và nằm ngoài phạm vi.

1.5 Bố cục

Báo cáo (i) hệ thống hóa thiết kế mô-đun SyscallAD và làm rõ cơ chế phát hiện leo thang đặc quyền của nó; (ii) tái lập SyscallAD và đánh giá thực nghiệm trên DongTing; (iii) đối chiếu *trung thực* với số liệu công bố và rút ra bài học triển khai.

Phần còn lại được tổ chức như sau: Mục 2 trình bày cơ sở lý thuyết; Mục 3 tổng quan framework DeSFAM và điểm qua Mô-đun 1, 3; Mục 4 đi sâu vào kiến trúc và thuật toán của SyscallAD; Mục 5 phân tích cách SyscallAD phát hiện leo thang đặc quyền; Mục 6 mô tả thiết lập thực nghiệm; Mục 7 báo cáo kết quả quan sát; Mục 8 thảo luận; Mục 9 kết luận.

2 Cơ sở lý thuyết

2.1 Leo thang đặc quyền và thoát container trong Kubernetes

Trong cụm Kubernetes, các container/pod được cô lập mức logic bằng namespace, cgroup và capabilities, nhưng chia sẻ chung nhân của nút. Bề mặt nguy hiểm nhất là *leo thang đặc quyền* và *thoát container*: tận dụng lỗ hổng nhân hoặc cấu hình sai (capabilities dư thừa, mount host-path, ptrace, unshare/setns, nạp mô-đun nhân) để vượt ranh giới cô lập và chiếm quyền trên nút [6]. Đặc điểm chung: mọi con đường khai thác đều biểu hiện thành các *chuỗi syscall đặc quyền* đặc trưng (Bảng 1), nên phân tích syscall là điểm quan sát tự nhiên để phát hiện chúng.

Bảng 1: Một số họ lỗ hổng leo thang đặc quyền/thoát container và dấu vết syscall.

Lỗ hổng	Cơ chế	Syscall liên quan
PwnKit (CVE-2021-4034)	pkexec setuid + tải .so	execve, openat/mmap
Dirty Pipe (CVE-2022-0847)	ghi đè trang pipe	splice (kèm write)
runc escape (CVE-2019-5736)	ghi đè /proc/self/exe	openat, execve
Lạm dụng capability	nạp mô-đun/đặc quyền	init_module, capset, mount

2.2 Giám sát lời gọi hệ thống cho container

Các cơ chế truyền thống (seccomp, AppArmor, SELinux) lọc theo *chính sách tĩnh* nên: (1) không thích nghi với hành vi thời gian chạy; (2) thiếu nhận thức về *chuỗi syscall* nên mù với tấn công nhiều bước; và (3) tốn chi phí cấu hình thủ công ở quy mô lớn. eBPF cho phép nạp logic giám sát/thực thi an toàn vào nhân tại các điểm vào syscall với chi phí thấp [10], và là nền tảng để thu thập luồng syscall mà một mô-đun phát hiện bất thường cần. Trọng tâm của báo cáo là chính *lớp phát hiện* (mô-đun SyscallAD), độc lập với cơ chế thu thập cụ thể; việc thu thập syscall thời gian thực được xem là đầu vào cho sẵn.

2.3 Phát hiện xâm nhập dựa trên chuỗi syscall

Hướng phát hiện bất thường dựa trên chuỗi syscall có lịch sử lâu dài, khởi nguồn từ “a sense of self” [11, 12] và các mô hình thay thế [13], cùng cảnh báo kinh điển về tấn công bất chước [14]. Các hướng hiện đại gồm phương pháp ngữ nghĩa [15], túi lời gọi (BoSC) cho container [16], và học sâu một lớp/biến phân như DAGMM [17], Deep SVDD [18], HIDS học sâu [19]. Riêng cho container có các đánh giá phát hiện bất thường [20], ngữ cảnh hóa syscall [21] và tập hợp Random/Isolation Forest [22]. SyscallAD nằm trong dòng này: hợp nhất VAE với iForest và bổ sung khớp mẫu lành tính bằng PrefixSpan.

2.4 Công trình gần đây (2023–2026)

Các nghiên cứu mới cho thấy SyscallAD nằm trong một không gian thiết kế *đang hoạt động sôi nổi*. Ở lớp thực thi, lọc syscall bằng eBPF có trạng thái đã vượt sec-comp tĩnh nhờ nhận thức ngữ cảnh và đối số [23]; các agent eBPF thời gian thực như eBPF-PATROL phát hiện trực tiếp leo thang đặc quyền, thoát container và reverse shell từ luồng syscall [24]. Ở lớp phát hiện bất thường, *chính công thức* VAE + iForest của SyscallAD được tái sử dụng và mở rộng: FedMon đưa nó lên học liên kết đa cụm Kubernetes [25], còn LIGHT-HIDS ghép trích đặc trưng DeepSVDD với bộ phát hiện *một lớp* bằng Rừng cô lập [26]; mô hình ngôn ngữ trên chuỗi syscall (LSTM/Transformer) được dùng cho phát hiện *tính mới* [27]. Ngược lại, các khảo sát gần đây [28, 29] chỉ ra rằng mô hình Transformer/LLM tuy nắm ngữ cảnh tốt hơn nhưng *chi phí và độ trễ cao*, khó dùng để chặn nội tuyến thời gian thực—cũng cố lựa chọn lối nhẹ VAE + iForest. Cuối cùng, các phương pháp đối kháng (tiềm seqGAN, bắt chước) cho thấy bộ phát hiện theo chuỗi vẫn có thể bị né tránh, và đề xuất hướng tăng cường độ bền vững [30]—một mối đe dọa mà chuyên đề bàn ở Mục 8.

2.5 Bộ mã hóa biến phân (VAE) và Rừng cô lập (iForest)

Hai khối học máy lõi của SyscallAD:

- **VAE** [31] học phân phối ẩn của dữ liệu *bình thường*; điểm bất thường là *lỗi tái cấu trúc*: mẫu lệch khỏi phân phối bình thường được tái tạo kém nên có lỗi lớn. VAE phù hợp với học một lớp (chỉ cần dữ liệu lạnh tính), thích hợp cho phát hiện tấn công zero-day.
- **iForest** [32] cô lập ngoại lệ bằng phân vùng ngẫu nhiên đệ quy: điểm hiếm bị cô lập ở độ sâu nông hơn, cho điểm bất thường cao. iForest bổ trợ VAE ở khía cạnh *ngoại lệ thống kê* mà lỗi tái cấu trúc có thể bỏ sót.

2.6 Bộ dữ liệu DongTing

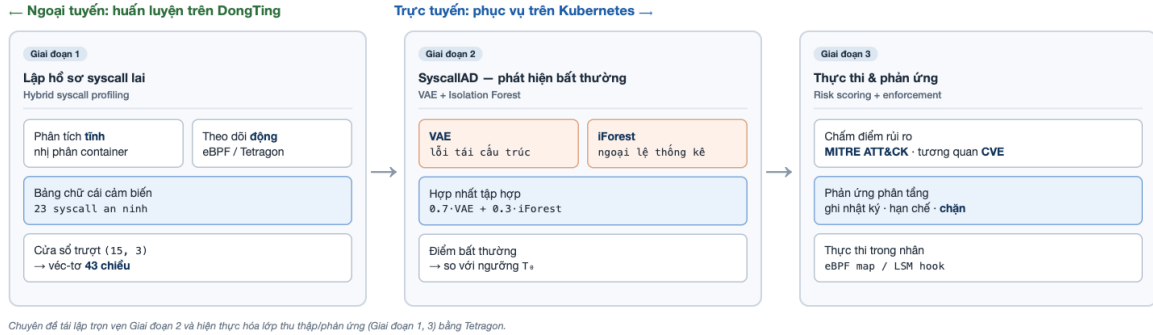
DongTing [2] là bộ dữ liệu quy mô lớn gồm các chuỗi syscall của nhân Linux được gán nhãn bình thường/độc hại (theo các bản vá lỗi syzkaller), thích hợp để huấn luyện và kiểm chứng bộ phát hiện bất thường. Đây cũng là nguồn dữ liệu mà SyscallAD trong DeSFAM sử dụng; báo cáo này dùng DongTing làm nguồn dữ liệu ngoại tuyến để tái lập và đánh giá. Chuỗi bình thường lưu dưới dạng *ID syscall*; chuỗi tấn công lưu dưới dạng *tên syscall*; bảng phân chia (Baseline.xlsx) định nghĩa các tập DTDS-train / DTDS-validation / DTDS-test.

3 Tổng quan framework DeSFAM

3.1 Kiến trúc ba mô-đun

DeSFAM (*Dynamic eBPF-Driven Syscall Filtering and Anomaly Mitigation*) [1] là một framework bảo mật thời gian thực, nhẹ cho container, thực thi nguyên tắc *đặc quyền*

tối thiểu ở tầng syscall nhằm giảm thiểu leo thang đặc quyền, thoát container và chèn syscall. DeSFAM hoạt động theo ba mô-đun (giai đoạn) tích hợp (Hình 1): lập hồ sơ/danh sách truy cập syscall ở không gian người dùng, phát hiện bất thường, và thực thi trong nhân.



Hình 1: Kiến trúc ba mô-đun của DeSFAM. Báo cáo này đặt trọng tâm vào Mô-đun 2 (SyscallAD)—phát hiện bất thường syscall; Mô-đun 1 và Mô-đun 3 chỉ được điểm qua.

3.2 Mô-đun 1 — Lập danh sách truy cập syscall lai và hồ sơ seccomp (điểm qua)

Mô-đun 1 sinh một danh sách truy cập syscall tối thiểu, dành riêng cho từng container, bằng cách kết hợp ba nguồn: (i) *phân tích tĩnh*—dịch ngược binary ELF (`objdump`, `readelf`) và dựng đồ thị lời gọi để rút tập syscall tĩnh S_{static} ; (ii) *lập hồ sơ động*—gắn eBPF probe dưới các khối lượng công việc chuẩn để thu tập syscall động S_{dynamic} ; (iii) *lọc theo CVE*—trích tên syscall rủi ro cao từ mô tả CVE (NVD) bằng NLP để tạo S_{blocked} . Danh sách cuối cùng được biên dịch thành hồ sơ seccomp và nạp động qua `policy_loader`. *Mô-đun này nằm ngoài trọng tâm báo cáo và không được hiện thực hóa lại*; ở đây chỉ ghi nhận rằng nó cung cấp lớp phòng thủ tĩnh, hỗ trợ cho lớp phát hiện động của Mô-đun 2.

3.3 Mô-đun 3 — Thực thi thích ứng và quản lý chính sách (điểm qua)

Mô-đun 3 nhận các chuỗi bị gắn cờ từ SyscallAD và quyết định hành động bằng *điểm rủi ro* tổng hợp từ: ánh xạ syscall sang kỹ thuật MITRE ATT&CK, mức nghiêm trọng tích lũy, và khớp mẫu khai thác CVE. Theo ngưỡng rủi ro, hệ thống *chặn* / *giới hạn tốc độ* / *ghi nhật ký*; việc chặn được thực thi trong nhân bằng eBPF gắn vào LSM hook (`bpf_override_return(-EPERM)`). DeSFAM còn có vòng tự phục hồi cập nhật lại danh sách seccomp khi một syscall bị chặn lặp lại. *Mô-đun này cũng nằm ngoài trọng tâm*: báo cáo dừng ở lớp phát hiện, còn việc chấm điểm rủi ro và chặn nội tuyến được nêu là hướng mở rộng.

3.4 Vị trí của SyscallAD và lý do chọn làm trọng tâm

SyscallAD (Mô-đun 2) là *trái tim phát hiện* của DeSFAM: Mô-đun 1 chỉ thu hẹp bề mặt tấn công theo chính sách tĩnh, còn Mô-đun 3 chỉ *phản ứng* dựa trên cờ mà SyscallAD phát ra. Nói cách khác, năng lực “nhận biết tấn công” của cả framework hội tụ ở SyscallAD. Đây cũng là phần mang tính *học máy* và *khả chuyển* nhất—độc lập với cơ chế thực thi của một nhân cụ thể—nên phù hợp nhất để nghiên cứu, tái lập và đánh giá định lượng. Vì vậy báo cáo dành toàn bộ Mục 4 và 5 cho mô-đun này.

4 Mô-đun SyscallAD — kiến trúc và thuật toán

Đây là phần trọng tâm của báo cáo. SyscallAD là một mô-đun phát hiện bất thường *kết hợp*, biến luồng syscall thô thành các véc-tơ đặc trưng, chấm điểm bằng tập hợp VAE + iForest và phát cờ khi điểm vượt ngưỡng. Mục này trình bày tuần tự toàn bộ đường ống đó.

4.1 Tổng quan đường ống phát hiện

Luồng xử lý của SyscallAD gồm năm bước nối tiếp:

luồng syscall → cửa sổ trượt → trích đặc trưng → VAE + iForest
→ hợp nhất & ngưỡng → cờ bất thường.

Mô hình được huấn luyện *chỉ trên dữ liệu bình thường* (học một lớp), nên không cần nhãn tấn công và có khả năng phát hiện cả mối đe dọa chưa từng thấy. Hình 2 minh họa toàn bộ đường ống.



Hình 2: Đường ống phát hiện của SyscallAD: lọc theo bảng chữ cái syscall an ninh → cửa sổ trượt (15, 3) → véc-tơ 43 chiều, chấm điểm song song bằng VAE (lỗi tái cấu trúc) và Isolation Forest (độ sâu cô lập), hợp nhất robust ($\alpha = 0,7$), so với ngưỡng T_0 để phát cờ. Cả VAE và iForest đều huấn luyện *chỉ trên cửa sổ lành tính*.

4.2 Thu thập, tiền xử lý và “bảng chữ cái syscall an ninh”

SyscallAD đọc một chuỗi định danh syscall theo tiến trình. Một quyết định thiết kế *then chốt* là **bảng chữ cái syscall an ninh**: giới hạn không gian đặc trưng về **23 syscall liên quan an ninh**—những lời gọi gắn với leo thang đặc quyền và thoát container (`execve`, `clone`, `setuid/setgid/capset`, `openat`, `unlinkat`, `renameat2`, `mount`, `unshare`, `setns`, `pivot_root`, `socket`, `connect`, `bind`, `mprotect`, `mmap`, `prctl`, `memfd_create`, `splice`, `ptrace`, `init/finit_module`, `bpf`—*không* gồm các lời gọi mật độ cao `read/write/close/futex`). Đây là một bước *chọn lọc đặc trưng theo miền*: tập trung bộ phát hiện vào hành vi đặc quyền và loại bớt nhiều nền. Khâu huấn luyện lọc mỗi chuỗi DongTing về *đúng* tập này *trước* khi cắt cửa sổ (xem hệ quả ở Mục 7).

4.3 Trích đặc trưng theo cửa sổ trượt

Chuỗi syscall được cắt thành các *cửa sổ trượt* chồng lấp (độ dài $L = 15$, bước nhảy 3—giá trị mặc định theo công trình tham chiếu). Mỗi cửa sổ được biểu diễn thành một véc-tơ **43 chiều** (Hình 3):

$$[\text{freq}_8 \mid \text{disc}_{15} \mid \text{stats}_8 \mid \text{cat}_{10} \mid \text{prefixspan}_2].$$

- **freq (8)**: tần suất trong cửa sổ của các syscall phổ biến trong bảng chữ cái.
- **disc (15)**: *hiện diện nhị phân* của các syscall đặc quyền hiếm (`ptrace`, `splice`, `unshare`, `setns`, `mount`, `bpf`, `init/finit_module`, `capset`, `memfd_create`...). Đây là *tín hiệu cốt lõi cho leo thang đặc quyền* (Mục 5) và bền với pha loãng: một `ptrace` đơn lẻ vẫn bật bit dù lẫn giữa nhiều syscall thông thường.
- **stats (8)**: entropy, số phần tử duy nhất, độ dài, tần suất lớn nhất... tóm tắt cấu trúc cửa sổ.
- **cat (10)**: tần suất chuẩn hóa theo nhóm chức năng (tiến trình, tệp, bộ nhớ, mạng, tín hiệu, thời gian, IPC, an ninh, sự kiện I/O, khác)—“chữ ký hành vi” theo danh mục.
- **prefixspan (2)**: [cờ-khớp, độ-dài-khớp/ L] từ việc đối sánh cửa sổ với một *Cơ sở dữ liệu mẫu lành tính* (Mục 4.4).
- **Đặc trưng thời gian (Δt)** bị *tắt* vì DongTing (`strace -v -f`) không có dấu thời gian theo từng syscall (`has_temporal=false`).

Chuẩn hóa bằng `RobustScaler` với phân vị (1,99), *chỉ* khớp trên cửa sổ huấn luyện bình thường để tránh rò rỉ dữ liệu tấn công.

4.4 Cơ sở dữ liệu mẫu lành tính bằng PrefixSpan

Để giảm dương tính giả, SyscallAD dùng PrefixSpan [33]—một thuật toán khai phá mẫu tuần tự—trên tập con *lành tính* để rút ra các chuỗi con syscall thường gặp, hợp lệ. Cơ sở dữ liệu mẫu này cho phép: (1) khớp sớm các cửa sổ “đã biết là tốt”; (2) bỏ qua chấm điểm sâu cho cửa sổ khớp, giảm chi phí; (3) cung cấp khả năng truy xuất ngữ nghĩa cho quyết định. Kết quả khớp được mã hóa thành hai đặc trưng `prefixspan` ở trên.



Hình 3: Kỹ thuật đặc trưng của SyscallAD: từ chuỗi syscall thô → lọc theo bảng chữ cái syscall an ninh → cửa sổ trượt (15, 3) → véc-tơ 43 chiều.

4.5 Mô hình thành phần

Bộ mã hóa biến phân (VAE). Kiến trúc encoder–decoder ($32 \rightarrow 8 \rightarrow 32$), `hidden_dim=32`, `latent_dim=8`, huấn luyện *chỉ* trên cửa sổ bình thường bằng cách tối thiểu hóa lỗi tái cấu trúc (MSRE). Điểm bất thường là MSRE; cửa sổ lệch phân phối bình thường có MSRE lớn. Để xử lý hiện tượng *sụp đổ hậu nghiệm* (posterior collapse), VAE được huấn luyện đa hạt giống (seed 0,1,2) và giữ hạt giống có AUC kiểm định tốt nhất.

Rừng cô lập (iForest). 300 cây, `contamination=0,02`, cũng huấn luyện trên dữ liệu bình thường. iForest cho điểm cao với các cửa sổ *hiếm về mặt thống kê* mà VAE có thể bỏ sót.

4.6 Hợp nhất tập hợp

Điểm cuối cho mỗi cửa sổ W_i là tổ hợp tuyến tính có trọng số của hai điểm thành phần (đã chuẩn hóa robust):

$$A(W_i) = \alpha \cdot \text{RS}(A_{\text{VAE}}(W_i)) + (1 - \alpha) \cdot \text{RS}(A_{\text{iForest}}(W_i)), \quad \alpha = 0,7. \quad (1)$$

Trọng số $\alpha = 0,7$ ưu tiên VAE; theo công trình tham chiếu, giá trị này được chọn qua phân tích độ nhạy trên $\alpha \in \{0; 0,25; 0,5; 0,7; 0,85; 1\}$.

4.7 Chọn ngưỡng

Biến quyết định để gán cờ là *điểm hợp nhất* $A(W_i)$ ở phương trình (1), nên mọi ngưỡng dưới đây đều áp lên A (không phải lên riêng lỗi tái cấu trúc của VAE). Báo cáo phân biệt hai khái niệm ngưỡng:

- **Ngưỡng ngoại tuyến T_0 (không rò rỉ):** chọn *chỉ* trên tập kiểm định từ phân phối điểm A của cửa sổ lạnh tính (mặc định: phân vị 99,5, tương ứng định hướng ưu tiên precision của công trình tham chiếu); tập kiểm tra *không* tham gia chọn ngưỡng. Đây là ngưỡng được dùng cho mọi kết quả trong báo cáo.

- **Ngưỡng động EMA (theo DeSFAM):** công trình tham chiếu cập nhật ngưỡng liên tục bằng Trung bình động hàm mũ. Báo cáo trình bày cơ chế này để đầy đủ, nhưng phân tái lập dùng ngưỡng cố định để dễ đối chiếu:

$$T_{t+1} = \max(T_{\min}, \beta T_t + (1 - \beta)A(W_i)), \quad \beta = 0,9, \quad T_{\min} \text{ là chặn dưới.} \quad (2)$$

4.8 Thuật toán phát hiện

Toàn bộ đường ống suy luận được tóm tắt trong Thuật toán 1.

Algorithm 1: Phát hiện bất thường syscall thời gian thực (SyscallAD)

Input: luồng syscall; VAE và iForest đã huấn luyện; bộ chuẩn hóa; ngưỡng T_0
Output: cờ bất thường cho mỗi cửa sổ
khởi tạo cửa sổ trượt $W \leftarrow \emptyset$; $T \leftarrow T_0$;
while luồng syscall còn hoạt động **do**
 thu syscall s_t tại thời điểm t ;
 lọc theo bảng chữ cái syscall an ninh; thêm s_t vào W ;
 if W đã đầy ($|W| = L$) **then**
 $f \leftarrow \text{TrichDacTrung}(W)$; // véc-tơ 43 chiều, đã chuẩn hóa
 $A_{\text{VAE}} \leftarrow \text{LoiTaiCauTruc}(\text{VAE}, f)$;
 $A_{\text{iForest}} \leftarrow \text{DiemCoLap}(\text{iForest}, f)$;
 $A \leftarrow 0,7 \cdot \text{RS}(A_{\text{VAE}}) + 0,3 \cdot \text{RS}(A_{\text{iForest}})$;
 if $A > T$ **then**
 gắn cờ W là bất thường; chuyển cho Mô-đun 3 (ngoài phạm vi);
 else
 cập nhật ngưỡng động: $T \leftarrow \max(T_{\min}, \beta T + (1 - \beta)A)$;
 trượt W về phía trước theo bước nhảy;

5 Cơ chế phát hiện leo thang đặc quyền của SyscallAD

Mục này nói mô-đun SyscallAD với ngữ cảnh đề tài: vì sao một bộ phát hiện bất thường syscall lại nắm bắt được hành vi leo thang đặc quyền, và đặc trưng nào mang tín hiệu đó.

5.1 Ánh xạ syscall đặc quyền sang kỹ thuật leo thang

Mọi con đường leo thang đặc quyền/thoát container đều phải gọi một số syscall đặc quyền mà khối lượng công việc lành tính hầu như không dùng. Bảng 2 ánh xạ các syscall đặc quyền sang nhóm kỹ thuật tấn công và ví dụ CVE; Hình 4 minh họa trực quan.

Bảng 2: Ánh xạ syscall đặc quyền → hành vi leo thang → kỹ thuật MITRE ATT&CK / ví dụ CVE.

Syscall	Hành vi	MITRE ATT&CK / ví dụ
ptrace	tiêm/điều khiển tiến trình khác	T1055 (Process Injection)
execve+tải .so	lạm dụng nhị phân setuid	T1548; PwnKit (CVE-2021-4034)
setuid/capset	đặt UID/capability đặc quyền	T1548 (Abuse Elevation Control)
unshare/setns	thao tác namespace, vượt cô lập	T1611 (Escape to Host)
mount/pivot_root	gắn/đổi gốc hệ tệp	T1611; thoát qua host-path
openat+execve	ghi đè /proc/self/exe	T1611; thoát runc (CVE-2019-5736)
init_module/finit_module	nạp mô-đun nhân	T1014 (Rootkit)
bpf	nạp chương trình eBPF đặc quyền	T1068; lạm dụng CAP_BPF
splice	ghi đè bộ đệm pipe	T1068; Dirty Pipe (CVE-2022-0847)

Lưu ý: (i) PwnKit và thoát runc không tự gọi setuid; dấu vết của chúng là execve (vào pkexec/ghi đè /proc/self/exe) kèm tải thư viện/ghi tệp—bảng này gắn chúng với syscall thực sự xuất hiện trong vết, nhất quán với Bảng 1. (ii) Một số khai thác dùng syscall ngoài bảng chữ cái syscall an ninh (ví dụ Dirty Pipe dùng cả write); khi đó SyscallAD vẫn bắt được nhờ bit hiện diện của syscall đặc quyền nằm trong bảng chữ cái (splice).



Hình 4: Ánh xạ các syscall đặc quyền (đặc trưng disc) sang nhóm hành vi leo thang đặc quyền và ví dụ CVE thực. Bit hiện diện của một syscall đặc quyền là tín hiệu phát hiện cốt lõi của SyscallAD.

5.2 Vì sao đặc trưng `disc` là tín hiệu cốt lõi

Trong một cửa sổ diễn hình của khối lượng hành tính, các syscall đặc quyền gần như không xuất hiện; còn trong cửa sổ chứa hành vi leo thang, chúng xuất hiện—dù chỉ *một lần*. Nếu dùng *tần suất*, một `ptrace` đơn lẻ lẫn giữa 14 lời gọi `openat` sẽ bị pha loãng tới mức gần như vô hình. SyscallAD vì thế mã hóa các syscall đặc quyền hiếm bằng **bit hiện diện nhị phân** (đặc trưng `disc`): bit bật ngay khi syscall xuất hiện, *không* phụ thuộc mật độ. Đây là lý do thiết kế khiến tấn công *bền với pha loãng* và là cầu nối trực tiếp giữa “phát hiện bất thường” và “phát hiện leo thang đặc quyền”.

VAE củng cố tín hiệu này: vì chỉ học phân phối của cửa sổ *bình thường* (nơi các bit `disc` hầu như bằng 0), một cửa sổ có bit `disc` bật sẽ nằm xa phân phối đã học và bị tái cấu trúc kém $\Rightarrow$ lỗi (điểm bất thường) cao. iForest hỗ trợ bằng cách cô lập tổ hợp syscall hiếm.

5.3 Ví dụ minh họa một chuỗi tấn công

Xét một chuỗi leo thang đặc quyền diễn hình `clone` $\rightarrow$ `setuid` $\rightarrow$ `execve` (kèm một `ptrace` dò xét). Khi đi qua đường ống:

1. Cửa sổ chứa các lời gọi này bật các bit `disc` tương ứng (`setuid`, `ptrace`) và lệch “chữ ký danh mục” cat về phía nhóm *process/security*.
2. Cửa sổ không khớp mẫu hành tính PrefixSpan $\Rightarrow$ đặc trưng `prefixspan` báo “không khớp”.
3. VAE trả lỗi tái cấu trúc cao; iForest cô lập cửa sổ ở độ sâu nông.
4. Điểm hợp nhất $A(W_i)$ vượt ngưỡng $\Rightarrow$ cửa sổ bị gán cờ.

Ngược lại, một cửa sổ hành tính (ví dụ truy vấn cơ sở dữ liệu) chỉ gồm các syscall phổ biến, khớp mẫu PrefixSpan và được VAE tái cấu trúc tốt $\Rightarrow$ điểm thấp, không gán cờ. Khoảng cách điểm giữa hai loại cửa sổ chính là cơ sở để đặt ngưỡng vận hành (kết quả định lượng ở Mục 7).

6 Thiết lập thực nghiệm

Mục này mô tả cách tái lập SyscallAD để *chạy lại* thực nghiệm. Số liệu kết quả được báo cáo ở Mục 7 *chỉ* lấy từ lần chạy lại trên bộ dữ liệu DongTing đầy đủ (chưa phải số liệu trong mục này).

6.1 Nguyên tắc: mọi công cụ chạy bằng Docker

Để bảo đảm khả năng tái lập và không làm “bẩn” máy chủ, toàn bộ chuỗi công cụ (huấn luyện, đánh giá, biên dịch báo cáo) đều chạy trong container. Quy trình huấn luyện và đánh giá được đóng gói bằng `docker-compose` và điều phối bởi `Experiment/run_experiment.sh`.

6.2 Huấn luyện ngoại tuyến SyscallAD

Một nguồn chân lý duy nhất là `train.py` (mọi hàm học máy, không vẽ đồ thị); số tay `train_dongting.ipynb` nhập lại từ `train.py` và bổ sung đồ thị + bình luận. Hai chế độ luôn đồng bộ.

```
cd DeSFAM/training
# Chạy nhanh, không tương tác:
docker compose -f docker-compose.pipeline.yml run --rm train
# Hoac tương tác (Jupyter Lab :8888):
docker compose -f docker-compose.pipeline.yml --profile interactive up jupyter
```

Đầu ra (lưu vào `../model/`): bộ chuẩn hóa đặc trưng, mô hình iForest, trọng số encoder/decoder VAE, các bộ chuẩn hóa điểm thành phần, tham số tập hợp ($\alpha = 0,7$), từ vựng đặc trưng (`fe_report.json`) và các chỉ số.

6.3 Cấu hình thí nghiệm (khai báo trước)

Bảng 3 là các *siêu tham số đã chọn* (cố định trước khi chạy); các đại lượng phụ thuộc dữ liệu (số chuỗi, số cửa sổ mỗi tập) sẽ được điền từ lần chạy lại trên DongTing đầy đủ.

Bảng 3: Cấu hình thí nghiệm đã khai báo cho SyscallAD.

Tham số	Giá trị
Thí nghiệm	<code>dongting_15_3</code>
Cửa sổ trượt (độ dài, bước)	(15, 3)
Số chiều đặc trưng	43
VAE <code>latent_dim</code> / <code>hidden_dim</code>	8 / 32
iForest <code>contamination</code>	0,02
Trọng số hợp nhất α	0,7
Bảng chữ cái syscall an ninh	23 syscall an ninh (<code>SENSOR_ALPHABET=1</code>)
Đặc trưng thời gian	không (<code>has_temporal=false</code>)
Chiến lược ngưỡng	p995 (phân vị 99,5 lỗi kiểm định lành tính)
Số chuỗi DongTing (nạp được / công bố)	18 874 / 18 966
Số cửa sổ train (lành tính) / val / test	90 456 / 896 581 / 687 926
Số mẫu lành tính (PrefixSpan)	150

6.4 Giao thức đánh giá và tiêu chí tái lập thành công

Để tránh diễn giải tùy tiện sau khi có số liệu, báo cáo *cố định trước* giao thức đánh giá:

- **Đơn vị đánh giá:** mỗi *cửa sổ trượt* (15, 3) là một mẫu. Nhân cửa sổ kế thừa nhãn chuỗi DongTing chứa nó. *Cảnh báo phương pháp:* một chuỗi tấn công chứa nhiều cửa sổ “trông lành tính” (không có syscall đặc quyền nào) nhưng vẫn bị gán nhãn tấn công; điều này gây nhiễu nhãn và có thể thổi phồng/làm lệch AUC/AP/F1. Do đó báo cáo *đồng thời* báo cáo (a) chỉ số mức *cửa sổ* và (b) chỉ số mức *chuỗi* (một chuỗi bị gán cờ nếu có cửa sổ vượt ngưỡng), và nêu tỉ lệ cửa sổ tấn công thực sự chứa syscall đặc quyền.

- **Phân chia dữ liệu:** dùng đúng phân chia DTDS-train/validation/test của DongTing; huấn luyện trên cửa sổ *bình thường* của tập train, chọn ngưỡng trên tập validation, báo cáo số trên tập test. Nêu rõ rằng việc chọn hạt giống VAE theo AUC kiểm định *có dùng nhãn tấn công của tập validation*; tập test tuyệt đối không tham gia chọn mô hình/ngưỡng.
- **Chỉ số:** AUC và Average Precision (AP)—đo khả năng *xếp hạng* bất thường, độc lập ngưỡng; F1/Precision/Recall—đo *điểm chốt nhị phân* tại ngưỡng đã chọn. Báo cáo kèm *trung bình $\pm$ độ lệch chuẩn* qua các hạt giống thay vì một điểm số đơn lẻ.
- **Tiêu chí “tái lập đạt” (không neo theo số liệu công bố theo mô hình—vì DeSFAM *không* công bố bảng AUC/AP/F1 theo mô hình trên DongTing):** coi là đạt nếu (a) bộ trích đặc trưng dùng khi huấn luyện và khi suy luận sinh véc-tơ *trùng khít* (kiểm thử ngang hàng đạt); (b) khả năng *xếp hạng* cao và ổn định (AUC/AP rõ rệt trên mức ngẫu nhiên, ổn định qua hạt giống); (c) định hướng *precision cao* (ít báo động giả) được giữ. F1/recall phụ thuộc ngưỡng nên *không* dùng làm tiêu chí đạt/không; mọi đối chiếu với DeSFAM chỉ ở mức *định tính* (Mục 7).
- **Thí nghiệm phân định (bắt buộc):** để tách “phát hiện bất thường theo chuỗi” khỏi “chỉ phát hiện sự hiện diện của syscall đặc quyền”, báo cáo chạy một *ablation*: so AUC của mô hình đầy đủ với một bộ phân loại *chỉ dùng đặc trưng disc* (Mục 8.4). Nếu hai bên gần nhau, kết luận phải lùi về “phát hiện theo hiện diện là tái lập được; mô hình học máy bổ sung biên [đo được]”.

Đối sánh mã syscall giữa hai lớp. Trong DongTing, chuỗi lành tính lưu dưới dạng *ID số*, còn chuỗi tấn công lưu dưới dạng *tên syscall*. Báo cáo dùng `syscall_64.tbl` (bỏ dòng ABI x32) để ánh xạ tên $\leftrightarrow$ ID nhất quán và kiểm tra rằng cả 23 syscall trong bảng chữ cái khôi phục *giống hệt* từ cả hai cách lưu—nếu lệch, các bit `disc` sẽ bật bất đối xứng giữa hai lớp và tạo/triệt tiêu độ phân tách một cách giả tạo.

Trung thành với nguồn so với thích nghi. Bảng 4 tách bạch phần tái lập *đúng* công bố với phần *thích nghi có lý do*, giúp diễn giải đúng chênh lệch số liệu ở Mục 7.

Bảng 4: Trung thành với DeSFAM so với thích nghi trong bản tái lập.

Hạng mục	Trạng thái	Ghi chú
VAE+iForest, $\alpha = 0,7$, cửa sổ (15,3)	tái lập đúng	α kế thừa, không tinh chỉnh lại
Huấn luyện một lớp (chỉ lành tính)	tái lập đúng	
Đặc trưng thời gian (Δt)	thích nghi (tắt)	DongTing không có dấu thời gian
Bảng chữ cái syscall	thích nghi (23 lời gọi)	tập syscall an ninh liên quan leo thang
Ngưỡng	thích nghi (cố định)	dùng thay EMA để dễ đối chiếu
Mô-đun 1, 3	ngoài phạm vi	không hiện thực hóa lại

6.5 Dữ liệu tấn công và lành tính

Báo cáo đánh giá hoàn toàn *ngoại tuyến* trên DongTing: chuỗi *lành tính* đến từ bốn bộ kiểm thử hồi quy nhân Linux, chuỗi *tấn công* từ các chương trình kích hoạt lỗi (syzkaller) trên 26 phiên bản nhân—các chuỗi này phản ánh các họ khai thác leo thang đặc quyền/thoát container (ví dụ CVE-2021-4034, CVE-2022-0847, CVE-2019-5736). Không có thành phần phục vụ trực tiếp; mọi số liệu lấy từ tập kiểm tra DongTing.

7 Kết quả và quan sát

Ghi chú. Số liệu ngoại tuyến (Mục 7.1–2) lấy *trực tiếp* từ lần chạy lại trên bộ dữ liệu DongTing đầy đủ (Zenodo 6627050; 18 874/18 966 chuỗi nạp được; 17/06/2026), qua quy trình ở thư mục Experiment/ (results.json + rigorous_metrics.json). Kiểm tra nhất quán đạt: AUC tập hợp mức cửa sổ tính lại = 0,835 khớp results.json. Báo cáo giới hạn ở *đánh giá ngoại tuyến* trên DongTing (huấn luyện → kiểm tra → đánh giá).

7.1 Hiệu lực phát hiện ngoại tuyến trên DongTing

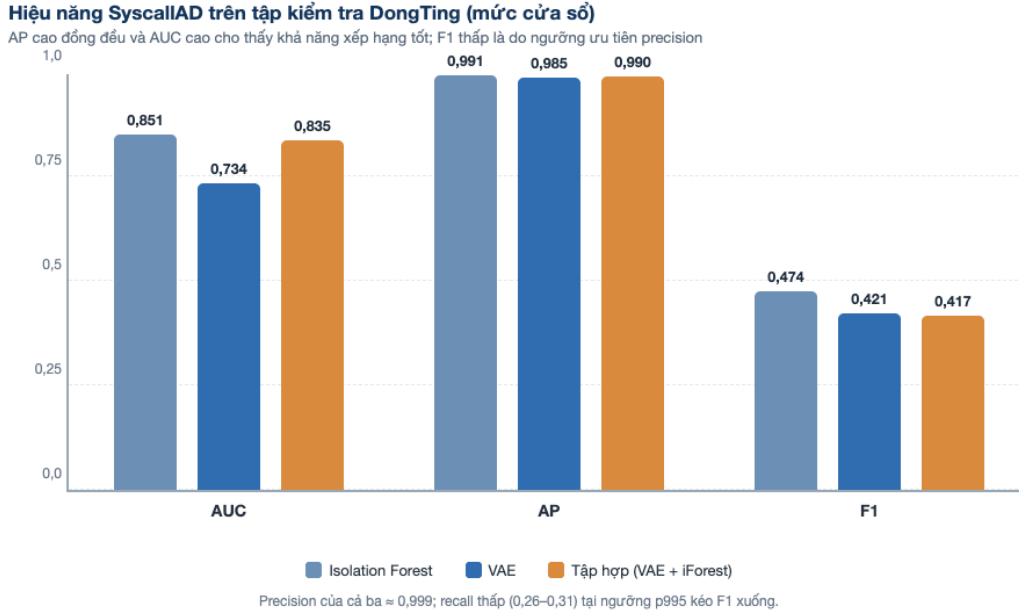
Bảng 5 trình bày hiệu năng trên tập kiểm tra DongTing (ngưỡng chọn trên tập kiểm định, chiến lược p995); Hình 5 trực quan hóa. Tập kiểm tra có 687 926 cửa sổ (2 305 chuỗi); tập kiểm định 896 581 cửa sổ (1 804 chuỗi).

Bảng 5: Hiệu năng SyscallAD trên tập kiểm tra DongTing đầy đủ (cửa sổ 15/3, 43 chiều). Báo cáo ở *cả* hai mức cửa sổ và chuỗi (Mục 6.4); hai hàng cuối là ablation “chỉ disc” (Mục 8.4).

Mô hình	AUC	AP	F1	Precision	Recall
iForest (cửa sổ)	0,851	0,991	0,474	0,999	0,311
VAE (cửa sổ)	0,734	0,985	0,421	0,999	0,267
Tập hợp (cửa sổ)	0,835	0,990	0,417	0,999	0,264
Tập hợp (chuỗi)	0,787	0,871	0,502	0,977	0,338
Ablation chỉ disc (cửa sổ)	0,875	0,990	—	—	—
Ablation chỉ disc (chuỗi)	0,678	—	—	—	—

VAE AUC kiểm định ổn định: $0,943 \pm 0,001$ qua 3 hạt giống $\{0, 1, 2\}$ (chọn hạt giống 1, 0,944); AUC tập hợp trên tập kiểm tra $0,835 \pm 0,000$ qua các hạt giống.

Quan sát. (i) *Xếp hạng*: lỗi phát hiện đạt AUC/AP cao và ổn định (tập hợp AUC 0,835, AP 0,990; AUC tập hợp gần như bất biến qua hạt giống, $\pm 0,000$). (ii) *Điểm chốt nhị phân*: tại ngưỡng p995 ưu tiên precision (0,999), recall mức cửa sổ thấp (0,264) nên F1 thấp (0,417); đúng như giao thức đã lường trước, chênh lệch F1 truy về *chiến lược ngưỡng* chứ không phải năng lực xếp hạng. (iii) *Cửa sổ so với chuỗi*: khi gộp lên mức chuỗi (gắn cờ nếu có cửa sổ vượt ngưỡng), recall tăng (0,338) và F1 tăng (0,502) dù AUC giảm nhẹ (0,787)—xác nhận rằng nhân mức cửa sổ làm lệch các chỉ số và cần báo cáo cả hai mức. (iv) *Ablation*: xem Mục 8.4 (kết quả có hệ quả với phản biện cốt lõi).



Hình 5: Hiệu năng mức cửa sổ của ba mô hình trên tập kiểm tra DongTing. AP cao đồng đều ($\approx 0,99$) và AUC cao (0,73–0,85) phản ánh khả năng *xếp hạng* tốt; F1 thấp (0,42–0,47) là hệ quả của ngưỡng p995 ưu tiên precision (recall 0,26–0,31).

7.2 Tham chiếu định tính tới DeSFAM

Một điểm cần được nêu rõ: bài báo gốc DeSFAM [1] **không công bố bảng AUC/AP/F1 theo từng mô hình** cho SyscallAD trên DongTing. Bài chỉ nêu vài con số *tổng hợp* (precision $\approx 94\%$, recall $\approx 90\%$, F1 $\approx 0,92$, AUC $\approx 0,94$), và những con số này thậm chí không hoàn toàn nhất quán trong chính bài (ví dụ phần văn bản ghi F1 của riêng VAE $\approx 0,84$ trong khi bảng lại ghi cao hơn). Vì vậy báo cáo *cố ý không* lập bảng “đối chiếu theo mô hình với số liệu công bố”—một bảng như thế sẽ là *gán nhầm* (các phiên bản trước từng dùng các giá trị 0,646/0,747/0,656 và mô tả là “số liệu bài báo”, đây là một sai sót đã được sửa). Ở đây chỉ tham chiếu *định tính*.

So với mức tổng hợp nói trên, bản tái lập đạt AUC/AP *cùng tầm cao* (tập hợp AUC 0,835, AP 0,990, precision 0,999) nhưng recall/F1 *thấp hơn hẳn* (0,264/0,417 mức cửa sổ). Khác biệt này quy về hai lựa chọn thiết kế đã nêu rõ: (i) ngưỡng p995 ưu tiên precision thay vì ngưỡng tối ưu F1 mà mức công bố 0,92 ngụ ý; (ii) bảng chữ cái 23 syscall + véc-tơ 43 chiều hẹp hơn thiết lập đầy đủ. Báo cáo *không* tuyên bố “đạt/vượt số liệu công bố”; kết luận giới hạn ở chỗ đã *tái hiện trung thành đường ống* và giữ được khả năng xếp hạng cùng định hướng ít báo động giả (xem tiêu chí ở Mục 6.4).

7.3 Cấu hình bảng chữ cái syscall an ninh

Toàn bộ kết quả trên dùng SENSOR_ALPHABET=1 (lọc DongTing về đúng 23 syscall an ninh *trước* khi cắt cửa sổ), cho AUC kiểm định VAE 0,944. Việc giới hạn về tập syscall an ninh là một bước *chọn lọc đặc trưng theo miền* (Mục 4.2), giúp bộ phát hiện tập trung vào hành vi đặc quyền. Phép đối chứng định lượng với chế độ SENSOR_ALPHABET=0 (toàn bộ vũ trụ syscall) được nêu là hướng phát triển (Mục 9).

8 Thảo luận

8.1 SyscallAD đạt được gì so với công trình tham chiếu

Về mặt thiết kế, SyscallAD tái hiện được *định hướng* của lõi phát hiện DeSFAM: một bộ phát hiện học một lớp (chỉ dùng dữ liệu lạnh tính) kết hợp lõi tái cấu trúc (VAE) với cô lập thống kê (iForest), trong đó các bit hiện diện syscall đặc quyền là tín hiệu trực tiếp cho leo thang đặc quyền. Việc đánh giá “tái lập thành công hay chưa” được căn theo tiêu chí đã cố định trước ở Mục 6.4 (ưu tiên khả năng xếp hạng AUC/AP và định hướng precision, không lấy chênh lệch F1 tại một ngưỡng làm phán quyết). Một điểm cần lưu ý: DeSFAM không công bố bảng AUC/AP/F1 theo mô hình trên DongTing (Mục 7.2), nên ở đây *không* có phán quyết “đạt/không đạt số liệu công bố theo mô hình”. Điều khẳng định được, một cách trung thực, là: bản tái lập đạt khả năng *xếp hạng* cao và ổn định (tập hợp AUC 0,835, AP 0,990, $\pm 0,000$ qua hạt giống) và giữ định hướng *ít báo động giả* (precision 0,999); còn recall/F1 thấp (0,264/0,417) là hệ quả của ngưỡng p995 và bảng chữ cái hẹp, không phải của năng lực xếp hạng. So với mức *tổng hợp* của DeSFAM (AUC $\approx 0,94$, F1 $\approx 0,92$), AUC cùng tầm nhưng F1/recall thấp hơn—chênh lệch quy về ngưỡng và cấu hình.

8.2 Ý nghĩa của khoảng cách ngưỡng và quản trị ngưỡng khi vận hành

Với phát hiện bất thường, một mô hình *xếp hạng* tốt (AUC/AP cao) vẫn có thể cho F1 thấp tại một ngưỡng cố định nếu ngưỡng ưu tiên precision. Điều này cho thấy giá trị của *hiệu chỉnh ngưỡng theo môi trường*: ngưỡng nên được căn theo phân phối điểm quan sát trong môi trường triển khai thay vì cố định từ tập kiểm định. Số liệu xác nhận điều này: tại ngưỡng p995, precision đạt 0,999 nhưng recall mức cửa sổ chỉ 0,264 (F1 0,417); gộp lên mức chuỗi, recall tăng lên 0,338 và F1 lên 0,502. Khoảng cách so với F1 tổng hợp của DeSFAM ($\approx 0,92$) vì thế quy về *chiến lược ngưỡng* (ưu tiên precision), không phải năng lực xếp hạng—vốn cao và ổn định (AUC 0,835, AP 0,990).

8.3 Lựa chọn thiết kế và phạm vi

Báo cáo giới hạn không gian đặc trưng về 23 syscall an ninh—một bước *chọn lọc đặc trưng theo miền*, tập trung bộ phát hiện vào hành vi đặc quyền. Đánh giá được thực hiện hoàn toàn ngoại tuyến trên DongTing; thành phần phục vụ trực tiếp cùng các Mô-đun 1 và 3 của DeSFAM (lập danh sách seccomp, chấm điểm rủi ro, chặn trong nhân) nằm ngoài trọng tâm và không được hiện thực hóa lại—báo cáo dừng ở mức phát hiện.

8.4 Phản biện về ảnh hưởng của việc thu hẹp bảng chữ cái syscall an ninh

Một phản biện đáng lưu ý cần được xem xét: vì không gian đặc trưng giới hạn ở 23 syscall an ninh và khối lượng lạnh tính hầu như không gọi syscall đặc quyền, có thể nghi ngờ rằng SyscallAD thực chất chỉ xác định *sự hiện diện của một syscall đặc quyền*

chứ không thực sự phát hiện *bất thường theo chuỗi*. Đây là một nguy cơ *những thiết kế* (confound) cần được thừa nhận. Ba lập luận làm giảm nhẹ—song không loại bỏ hoàn toàn—phản biện này: (i) không phải syscall nào trong bảng chữ cái cũng đặc quyền (execve, openat, socket, mmap xuất hiện dày trong cả hai lớp), nên mô hình vẫn phải học cấu trúc cửa sổ chứ không chỉ đọc một bit; (ii) đặc trưng cat/stats/prefixspan mã hóa phân bố và tính tuần tự, không chỉ hiện diện; (iii) một số kỹ thuật leo thang lạm dụng *chính* các syscall hợp lệ (ví dụ chuỗi execve bất thường), nơi bit hiện diện không đủ và mô hình chuỗi mới phân biệt được.

Kết quả thí nghiệm cắt bỏ. Thí nghiệm cắt bỏ “chỉ-disc” cho một kết luận phân đôi rõ ràng (Hình 6):

- **Ở mức cửa sổ, phản biện có cơ sở.** Bộ đếm bit disc (chỉ biểu thị sự hiện diện của syscall đặc quyền) đạt AUC 0,875, cao hơn cả tập hợp đầy đủ (0,835); 83,5% cửa sổ tấn công chứa ít nhất một syscall đặc quyền. Ở mức cửa sổ, SyscallAD về cơ bản *tương đương* một bộ phát hiện dựa trên hiện diện, phù hợp với phản biện nêu trên.
- **Ở mức chuỗi, mô hình có giá trị tăng thêm.** Khi gộp lên mức chuỗi, tập hợp đầy đủ đạt AUC 0,787 trong khi chỉ-disc chỉ đạt 0,678: cấu trúc cửa sổ mà VAE/iForest học được phân biệt tốt hơn đáng kể so với hiện diện đơn thuần, tại đơn vị vận hành thực (chuỗi/tiến trình).

Như vậy, tại đơn vị cửa sổ, năng lực phát hiện phần lớn quy về sự hiện diện của syscall đặc quyền; giá trị riêng của mô hình học máy chỉ bộc lộ ở mức chuỗi. Do đó báo cáo *không* khẳng định “cơ chế bất thường theo chuỗi vượt trội” một cách tổng quát mà giới hạn kết luận ở mức chuỗi, đồng thời đề xuất bổ sung khối lượng lạnh tính có gọi syscall đặc quyền hợp lệ (chẳng hạn trình gỡ lỗi, hoặc sidecar quan sát sử dụng bpf) để kiểm chứng chặt chẽ hơn.

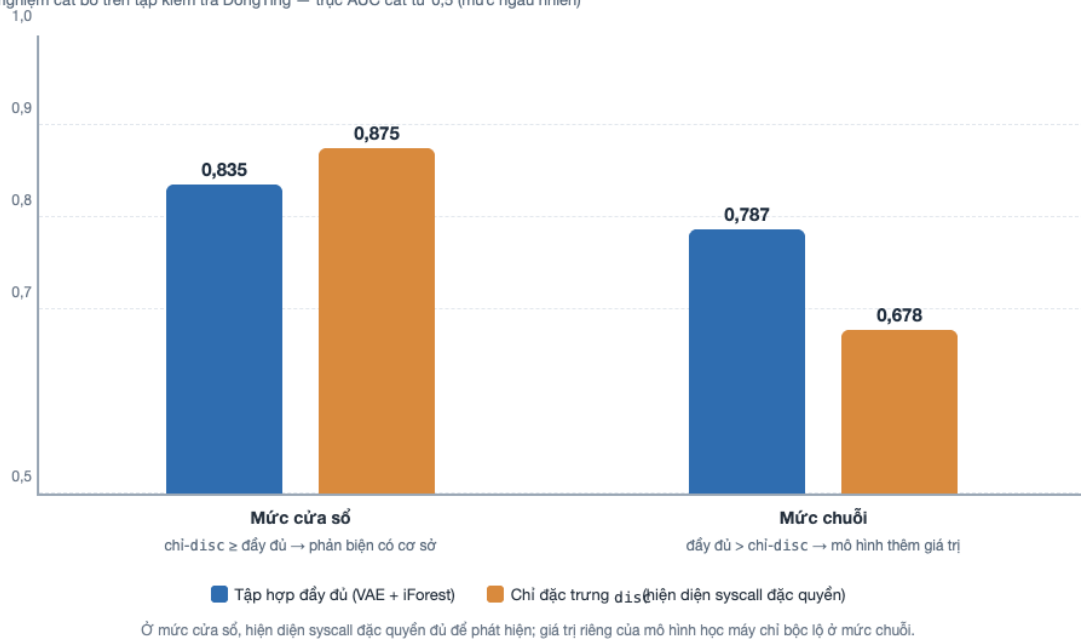
Đối soát mã hóa (cảnh báo từ kiểm toán). Kiểm toán đối soát mã hóa phát hiện 2/24 syscall trong bảng chữ cái (setgid, unlinkat) xuất hiện trong chuỗi lạnh tính (mã ID) nhưng *vắng mặt* trong chuỗi tấn công (mã tên). Đây có thể là bất đối xứng tự nhiên hoặc một *lệch ánh xạ tên → ID* cần điều tra (ví dụ tên thay thế như setgid32, unlink); nếu là lệch ánh xạ, các bit disc tương ứng sẽ bật bất đối xứng giữa hai lớp—một hạng mục cần kiểm tra trước khi diễn giải sâu hơn.

8.5 Hạn chế và đe dọa tính hợp lệ

- **Hợp lệ nội tại:** đánh giá dựa trên các chuỗi syscall của DongTing (chương trình kích hoạt lỗi nhân), không phải vết khai thác CVE thực thi đầu-cuối; kết quả phản ánh khả năng phân biệt trên dữ liệu này.
- **Hợp lệ ngoại tại:** DongTing là chuỗi lỗi nhân, có thể chưa đại diện đầy đủ cho hành vi container/khối lượng công việc sản xuất; cần mở rộng nguồn dữ liệu để khái quát hóa.

AUC theo đơn vị đánh giá: mô hình đầy đủ so với chỉ `disc`

Thí nghiệm cắt bỏ trên tập kiểm tra DongTing — trực AUC cắt từ 0,5 (mức ngẫu nhiên)



Hình 6: Thí nghiệm cắt bỏ trên tập kiểm tra DongTing: AUC của tập hợp đầy đủ (VAE + iForest) so với bộ phân loại chỉ dùng đặc trưng `disc`. Ở *mức cửa sổ*, chỉ-`disc` (0,875) ngang bằng hoặc cao hơn mô hình đầy đủ (0,835)—phản biện “đường tắt theo hiện diện” có cơ sở. Ở *mức chuỗi*, mô hình đầy đủ (0,787) vượt rõ chỉ-`disc` (0,678)—mô hình học máy đóng góp giá trị vượt trên hiện diện đơn thuần.

- **Hợp lệ kết luận:** mất cân bằng lớp lớn khiến AP dễ gây hiểu nhầm; cần báo cáo kèm AUC, precision, recall để tránh kết luận một chiều.
- **Hợp lệ cấu trúc:** đặc trưng thời gian bị tắt do DongTing không có dấu thời gian theo syscall; mô hình có thể thiếu một tín hiệu mà công trình tham chiếu tận dụng.
- **Nhãn mức cửa sổ:** việc kế thừa nhãn chuỗi xuống mọi cửa sổ (Mục 6.4) gán nhãn “tấn công” cho cả các cửa sổ lành tính bên trong chuỗi tấn công, làm lệch AUC/AP/F1; đây là lý do báo cáo kèm chỉ số mức chuỗi.
- **Tấn công thích nghi:** mô hình đe dọa giả định tấn công bộc lộ syscall đặc quyền; một kẻ tấn công biết bảng chữ cái 23 lời gọi có thể *bắt chước* (mimicry [14]) hành vi lành tính hoặc pha loãng để tránh bật bit disc. Công trình gần đây xác nhận bộ phát hiện theo chuỗi vẫn bị né tránh bằng tiêm đối kháng kiểu seqGAN và đề xuất hướng tăng cường độ bền vững [30]. Đây là giới hạn cố hữu của phát hiện dựa trên hiện diện và là hướng cần đánh giá đối kháng.

9 Kết luận và hướng phát triển

Báo cáo đã *ngiên cứu mô-đun phát hiện bất thường SyscallAD* của framework DeSFAM trong ngữ cảnh phát hiện tấn công leo thang đặc quyền dựa trên lời gọi hệ thống trong Kubernetes. Ứng với mục tiêu đề ra:

- **Hiểu thiết kế.** Báo cáo đã hệ thống hóa đường ống của SyscallAD (cửa sổ trượt $\rightarrow$ véc-tơ 43 chiều $\rightarrow$ VAE + iForest $\rightarrow$ hợp nhất $\rightarrow$ ngưỡng) và làm rõ *cơ chế* phát hiện leo thang đặc quyền: các syscall đặc quyền hiếm được mã hóa bằng bit hiện diện nhị phân (disc), bền với pha loãng, nằm xa phân phối lành tính mà VAE đã học nên cho điểm bất thường cao.
- **Tái lập & bằng chứng.** Trên DongTing đầy đủ, lỗi phát hiện đạt AUC tập hợp 0,835 (cửa sổ) / 0,787 (chuỗi), AP 0,990, precision 0,999, ổn định qua hạt giống ($\pm 0,000$). Vì DeSFAM không công bố bảng số theo mô hình trên DongTing, báo cáo *không* tuyên bố “đạt/vượt số liệu công bố”, chỉ tham chiếu định tính (AUC cùng tầm mức tổng hợp $\approx 0,94$; F1/recall thấp hơn do ngưỡng p995 + bảng chữ cái hẹp). Thí nghiệm cắt bỏ cho thấy ở mức cửa sổ năng lực phát hiện phần lớn quy về *hiện diện* syscall đặc quyền (chỉ-disc 0,875 $\geq$ tập hợp 0,835), còn giá trị riêng của mô hình bộc lộ ở *mức chuỗi* (0,787 so với 0,678)—một *bằng chứng sơ bộ* có giới hạn.
- **Nền tảng tái lập.** Toàn bộ quy trình được đóng gói bằng Docker và ràng buộc bằng kiểm thử ngang hàng, cho phép các nghiên cứu sau chạy lại và mở rộng một cách độc lập.

Về bài học phương pháp (độc lập với con số): với phát hiện bất thường, (a) việc chọn lọc đặc trưng theo miền (giới hạn về tập syscall an ninh) định hình mạnh kết quả, và (b) *đơn vị đánh giá* (cửa sổ so với chuỗi) cùng *tiêu chí đặt trước* quan trọng ngang chất lượng mô hình—bỏ qua chúng dễ dẫn tới kết luận sai lệch.

Hướng phát triển. (1) Chọn ngưỡng tối ưu F1 và báo cáo đầy đủ đường cong precision–recall; (2) phép đối chứng bảng chữ cái SENSOR_ALPHABET 0/1 để định lượng

đóng góp của bước chọn lọc đặc trưng—bộ công cụ đã sẵn trong `Experiment/`; (3) bổ sung đặc trưng thời gian khi có nguồn dữ liệu kèm dấu thời gian theo syscall (điều `DongTing` thiếu); (4) mở rộng nguồn dữ liệu và dùng vết khai thác CVE thực; (5) điều tra bất đối xứng mã hóa của `setgid/unlinkat` (Mục 8.4); (6) (xa hơn) kết nối khâu thực thi của DeSFAM để chuyển từ *phát hiện* sang *ngăn chặn*.

Tài liệu tham khảo

- [1] S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, “DeSFAM: An adaptive eBPF and AI-driven framework for securing cloud containers in real time,” *IEEE Access*, vol. 13, 2025.
- [2] G. Duan, Y. Fu, M. Cai, H. Chen, and J. Sun, “DongTing: A large-scale dataset for anomaly detection of the Linux kernel,” *Journal of Systems and Software*, vol. 203, p. 111768, Sep. 2023.
- [3] O. Jarkas, R. Ko, N. Dong, and R. Mahmud, “A container security survey: Exploits, attacks, and defenses,” *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025.
- [4] S. Sultan, I. Ahmad, and T. Dimitriou, “Container security: Issues, challenges, and the road ahead,” in *IEEE Access*, vol. 7, 2019, pp. 52 976–52 996.
- [5] R. Shu, X. Gu, and W. Enck, “A study of security vulnerabilities on docker hub,” in *Proceedings of the 7th ACM on Conference on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280.
- [6] T. Combe, A. Martin, and R. Di Pietro, “To docker or not to docker: A security perspective,” *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016.
- [7] NIST National Vulnerability Database, “CVE-2021-4034: Polkit local privilege escalation,” <https://nvd.nist.gov/vuln/detail/CVE-2021-4034>, 2022.
- [8] —, “CVE-2022-0847: Dirty pipe — linux kernel privilege escalation,” <https://nvd.nist.gov/vuln/detail/CVE-2022-0847>, 2022.
- [9] —, “CVE-2019-5736: Docker runc container escape,” <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>, 2019.
- [10] B. Gregg, “BPF performance tools: Linux system and application observability,” *Addison-Wesley Professional*, 2019, ISBN: 978-0-13-655482-1.
- [11] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, “A sense of self for Unix processes,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 1996, pp. 120–128.
- [12] S. A. Hofmeyr, S. Forrest, and A. Somayaji, “Intrusion detection using sequences of system calls,” *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.
- [13] C. Warrender, S. Forrest, and B. Pearlmutter, “Detecting intrusions using system calls: Alternative data models,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 1999, pp. 133–145.

- [14] D. Wagner and P. Soto, “Mimicry attacks on host-based intrusion detection systems,” in *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, 2002, pp. 255–264.
- [15] G. Creech and J. Hu, “A semantic approach to host-based intrusion detection systems using contiguous and discontiguous system call patterns,” *IEEE Transactions on Computers*, vol. 63, no. 4, pp. 807–819, 2014.
- [16] A. S. Abed, C. Clancy, and D. S. Levy, “Applying bag of system calls for anomalous behavior detection of applications in Linux containers,” in *Proceedings of the IEEE Globecom Workshops*, 2015.
- [17] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, “Deep autoencoding Gaussian mixture model for unsupervised anomaly detection,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [18] L. Ruff, R. A. Vandermeulen, N. Görnitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, “Deep one-class classification,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 4393–4402.
- [19] W. Haider, J. Ahmad, M. Nasim, Z. A. Baig, and A. A. Laghari, “Methods for host-based intrusion detection with deep learning,” *Digital Threats: Research and Practice*, vol. 2, no. 4, pp. 1–29, 2021.
- [20] P. Castanhel, P. Pinto, and F. Magalhães, “Taking a peek: An evaluation of anomaly detection using system calls for containers,” in *Proceedings of the IEEE International Symposium on Computers and Communications (ISCC)*, 2021.
- [21] M. El Khairi, D. Colle, F. De Turck, and B. Volckaert, “Contextualizing system calls in containers for anomaly-based intrusion detection,” in *Proceedings of the ACM Cloud Computing Security Workshop (CCSW)*, 2022.
- [22] Q. Bsoul, M. Al-Quraan, and F. Al-Turjman, “Ensemble of random and isolation forests for graph-based intrusion detection in containers,” in *Proceedings of the IEEE International Conference on Cyber Security and Resilience (CSR)*, 2022.
- [23] “Programmable system call security with eBPF,” 2023, preprint, arXiv:2302.10366.
- [24] “eBPF-PATROL: Protective agent for threat recognition and overreach limitation using eBPF in containerized and virtualized environments,” 2025, preprint, arXiv:2511.18155.
- [25] “FedMon: Federated eBPF monitoring for distributed anomaly detection in multi-cluster cloud environments,” 2025, preprint, arXiv:2510.10126.
- [26] “LIGHT-HIDS: A lightweight and effective machine learning-based framework for robust host intrusion detection,” 2025, preprint, arXiv:2509.13464.
- [27] “Language models for novelty detection in system call traces,” 2023, preprint, arXiv:2309.02206.

- [28] “Deep learning-based intrusion detection systems: A survey,” 2025, preprint, arXiv:2504.07839.
- [29] “Transformers and large language models for efficient intrusion detection systems: A comprehensive survey,” 2024, preprint, arXiv:2408.07583.
- [30] “An adversarial robust behavior sequence anomaly detection approach based on critical behavior unit learning,” 2025, preprint, arXiv:2509.15756.
- [31] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” in *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*, 2014.
- [32] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [33] J. Pei, J. Han, B. Mortazavi-Asl, J. Wang, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, “Mining sequential patterns by pattern-growth: The PrefixSpan approach,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 11, pp. 1424–1440, 2004.

A Phụ lục: Tái lập kết quả

Phụ lục cho phép một người đánh giá độc lập *chạy lại* thực nghiệm ngoại tuyến. Mọi công cụ chạy bằng *Docker*—không cần cài đặt trực tiếp lên máy chủ ngoài *Docker*.

A.1 Chuẩn bị dữ liệu DongTing đầy đủ

```
# DongTing trong kho chỉ là MAU. Tại ban DAY DU (Normal_data/, Abnormal_data/,
# Baseline.xlsx, syscall_64.tbl) và tro DATA_ROOT tôi thu mục giải nén.
export DATA_ROOT=/duong/dan/toi/dongting-full
```

A.2 Huấn luyện ngoại tuyến (sinh số cho Bảng 5)

```
cd DeSFAM/training
DATA_ROOT=$DATA_ROOT docker compose -f docker-compose.pipeline.yml run --rm train
# Quan sát chỉ số in ra; ghi lại kết quả + nhận xét vào Mục 7.
```

A.3 Kiểm tra tính nhất quán đặc trưng (train ↔ serve)

```
cd DeSFAM
docker compose -f training/docker-compose.pipeline.yml run --rm \
  train python -m pytest tests/test_feature_parity.py -q
```

A.4 Biên dịch báo cáo này (Docker)

```
cd ReportV2
./build.sh          # chạy XeLaTeX qua Docker texlive -> main.pdf
```